

Python Multithreading & The GIL

Threads, Race Conditions, Thread Locks, and the Global Interpreter Lock

1. Multithreading Basics

Python provides the `threading` module to work with threads.

```
import threading

# Creates a thread
t1 = threading.Thread(target=fun, args=())

# Starts thread execution
t1.start()

# Waits until the thread finishes execution
t1.join()
```

What happens if we remove `join()` ?

- The **main program may finish before the thread completes** its execution.
- Outputs from different threads **mix together**.
- **Why?** Because both threads run concurrently, switching back and forth.

Is Concurrency Always Good?

Sometimes it is helpful, but concurrent access without coordination causes extreme danger due to **Race Conditions**.

2. Race Conditions & Thread Safety

⚠ The Race Condition Problem

A Race Condition occurs when multiple threads attempt to access and modify shared data simultaneously.

Notebook Analogy: Two people are holding one notebook. Both try writing on Page 10 at the exact same moment. One person overwrites the other's work, causing data corruption.

The Solution: `threading.Lock()`

```
# Creates a Lock object
lock = threading.Lock()
```

Think of a Lock as creating a **key** or a **security guard** that controls and restricts access to a shared resource.

- `Lock()` → Creates a new lock instance.

- `lock` → Variable storing that lock object.



Real-Life Analogy: The Office Printer

Imagine there is only **one printer** in an office, and three employees want to print at once.

Without a Lock (Chaos)

```
Employee A --Employee B ----> Printer (Chaos!)
Employee C --/
```

Everyone prints simultaneously; pages get intermingled and ruined.

With a Lock (Order)

```
Employee A -> 🔒 Printer
Employee B -> Waiting...
Employee C -> Waiting...
```

Employee A finishes → 🔓 Unlocks → Employee B starts.

3. The GIL (Global Interpreter Lock)

What is the GIL?

Term	Meaning
Global	There is only one GIL per Python interpreter instance.
Interpreter	It belongs to the CPython interpreter (the standard Python implementation).
Lock	It acts as a lock, allowing only one thread at a time to execute Python bytecode.

Definition: The GIL is a lock inside CPython that ensures **only one thread executes Python bytecode at any single instant**.

Why does Python have the GIL?

Imagine Python managing objects in memory:

```
a = [1, 2, 3]
name = "Hari"
```

If two threads try to modify the same object simultaneously:

```
# Thread 1          # Thread 2
my_list.append(10)   my_list.append(20)
```

Without proper protection:

- Memory could become corrupted.
- Object reference counts become inconsistent.
- Python interpreter might crash.

The GIL simplifies CPython's memory management by guaranteeing thread-safe operations on internal objects.

Whiteboard Analogy (Teaching Concept)

Imagine a classroom with 5 students (Student A, B, C, D, E). There is **1 whiteboard** and **only 1 marker** ().

- Whoever holds the marker can write on the board. Everyone else must wait.
- Student A writes → gives marker to Student B → Student B writes → gives marker to C.
- **Everyone gets a turn, but only one student writes at a time.**

The marker IS the GIL!

4. Thread Execution Bottleneck on CPU Tasks

Suppose you create 3 threads:

```
import threading

t1 = threading.Thread(...)
t2 = threading.Thread(...)
t3 = threading.Thread(...)
```

Common Misconception: "Three threads means three tasks execute at the exact same time."

The CPython Reality: Python context-switches rapidly between threads so it *looks* simultaneous, but only one thread executes Python bytecode at any microsecond.

GIL Thread Scheduling Timeline

Time 1: Thread A [RUNNING]	Thread B [WAITING]	Thread C [WAITING]
Time 2: Thread A [WAITING]	Thread B [RUNNING]	Thread C [WAITING]
Time 3: Thread A [WAITING]	Thread B [WAITING]	Thread C [RUNNING]

CPU-Heavy Code Example

```
def calculate():
    for i in range(100_000_000):
        pass

t1 = threading.Thread(target=calculate)
t2 = threading.Thread(target=calculate)
```

Expected Parallelism

CPU Core 1 → Thread 1
CPU Core 2 → Thread 2

Actual Execution (Due to GIL)

Thread 1 → Thread 2 → Thread 1 → Thread 2 (Interleaved on 1 Core)

 **Conclusion: CPU-heavy work does NOT get faster with Python Multithreading. Use Multiprocessing instead!**